ECS 160 – Design Patterns

Instructor: Tapti Palit

Teaching Assistant: **Xingming Xu**

Recall: static

- “static” fields belongs to the class itself, not to any individual object / instance of that class
- Static attributes:
  - No need to instantiate an object of that class before calling or using said attributes
- Static methods can directly access static attributes without objects, but not non-static ones without
- Static methods can be accessed directly from any context

Design Patterns

- “Typical solutions to common problems”
  - Provides a shared vocabulary to describe them
- **Pattern** is the keyword here

Singleton

- One instance controlling everything
  - Ensure only one instance somehow?
- Use when you need strict control over global variables
- Key benefit: single controlled access
- NOTE: violates the single responsibility principle
  - Global access point problem + ensure class has a single instance

Why?

- One shared instance provides coordination
- Ex: logger
  - We want *shared* log files, *shared* states, *shared* behaviors
- Other cases:
  - Caches
  - Configuration manager
  - DB connection pool

Pros and Cons

- Pros:
  - Centralized global access
  - Shared resources
- Cons:
  - Can make testing difficult
  - Breaks pure OOP design

Implementation

1. Add a **private static field** to the class for storing the singleton instance.
2. Declare a **public static creation method** for getting the singleton instance.
3. Implement “lazy initialization” inside the static method. It should create a **new object** on its **first call** and put it into the static field.
   a. The method should always **return that instance** on all **subsequent** calls.
4. Make the constructor of the class private. The static method of the class will still be able to call the constructor, but not the other objects.
5. Go over the client code and replace all direct calls to the singleton’s constructor with calls to its static creation method.

Factory

- Create an object while hiding the creation logic from client
- Encapsulating creation logic

Why?

- Use when you don’t know exact types and dependencies of objects beforehand
- When you want to provide users of your library and framework with a way to extend its internal components

Factory Method

```
interface Notification { void send(String msg); }

class EmailNotification implements Notification {
    public void send(String msg) { System.out.println("Email: " + msg); }
}

class SmsNotification implements Notification {
    public void send(String msg) { System.out.println("SMS: " + msg); }
}
```

Factory Method

```
// Factory class
class NotificationFactory {
    public static Notification create(String channel) {
        return switch (channel) {
            case "EMAIL" -> new EmailNotification();
            case "SMS"   -> new SmsNotification();
            default      -> throw new IllegalArgumentException("Unknown type");
        };
    }
}
```

Factory Method

```
// Usage
class App {
    public static void main(String[] args) {
        Notification n = NotificationFactory.create("EMAIL");
        n.send("Hello from factory!");
    }
}
```

Implementation Details

1. Make all products follow the same interface. This interface should declare methods that make sense in every product.
2. Add an **empty factory method inside the creator class**. The return type of the method should match the common product interface.
3. **Replace** product **constructors** in creator's code with calls to factory method
4. Create a set of creator subclasses for each type of product listed in the factory method.
5. **Override** the factory method in the **subclasses** and extract the appropriate bits of construction code from the base method.

Adapter

- Integrate two incompatible interfaces
  - Client expecting one interface for a task
  - Existing task that can complete it, but expects another interface
- Where we don’t want to touch the original interface
- Analogy

Implementation

```
public class MediaAdapter implements MediaPlayer {
    private AdvancedMediaPlayer advancedPlayer = new AdvancedMediaPlayer();

    @Override
    public void play(String fileName) {
        if (fileName.endsWith(".mp4")) {
            advancedPlayer.playMp4(fileName);
        } else {
            System.out.println("Format not supported.");
        }
    }
}
```

Builder

- **Clearly construct** complex objects **step by step**

```
User user = new User.Builder()
    .name("Alice")
    .email("alice@mail.com")
    .age(30)
    .build();
```

Builder

- Extract object construction code out of its own class, move those to separate objects
- interface Builder is…
  - // Contains methods
- class Car is…
- class CarBuilder **implements** Builder is…
  - private field Car…
  - Implementing methods from the builder interface

Builder

```
class CarManualBuilder implements Builder is
    private field manual:Manual

    constructor CarManualBuilder() is
        this.reset()

    method reset() is
        this.manual = new Manual()

    method setSeats(...) is
        // Document car seat features.

    method setEngine(...) is
        // Add engine instructions.

    method getProduct():Manual is
        // Return the manual and reset the builder.
```